Proyecto: Bus Fever Party! *FPOO-C++ Edition.*

1. Introducción

El objetivo de este proyecto es diseñar e implementar el juego "Bus Fever Party! FPOO-C++ Edition". El estudiante deberá aplicar los fundamentos de la Programación Orientada a Objetos (POO) y los principios SOLID para crear una aplicación robusta, mantenible y escalable utilizando C++.

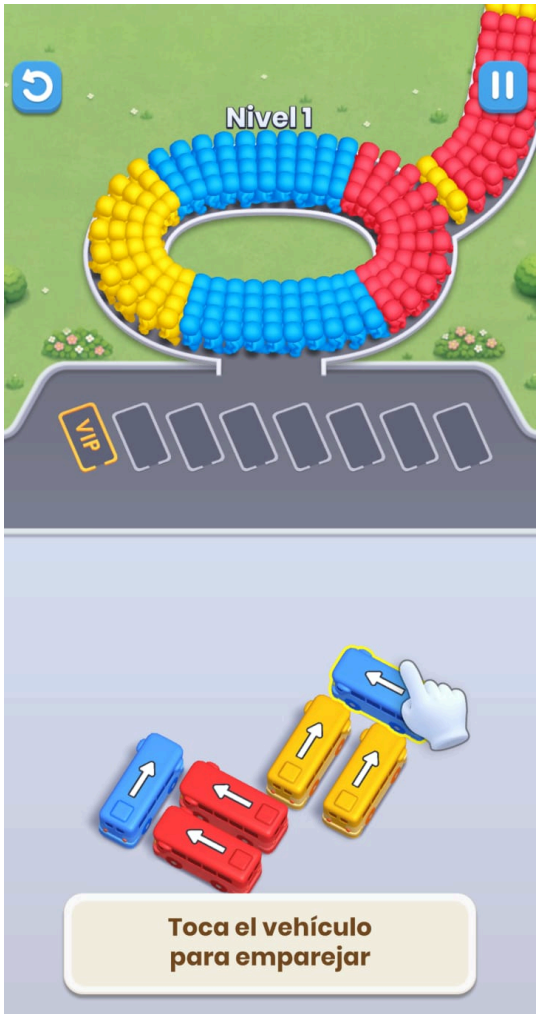


Figura 1: Interfaz de Bus Fever Party! Fuente: Captura de pantalla original del juego.

Para una comprensión del juego que se pretende replicar, se recomienda a los estudiantes descargar la aplicación original Bus Fever Party! desde la Google Play Store en sus dispositivos Android. Experimentar el juego de primera mano permitirá asimilar de forma intuitiva las reglas

de negocio del juego, el funcionamiento de la fila estática y la fila rotatoria de pasajeros y los desafíos espaciales que deberán ser traducidos a una arquitectura de software sólida y funcional.

El presente proyecto está diseñado para que el estudiante fortalezca y desarrolle sus capacidades de aprendizaje de forma autónoma, fomentando la investigación y la toma de decisiones técnicas independientes. Cabe destacar que el objetivo no es realizar una réplica exacta o visualmente idéntica de la aplicación móvil; lo que se busca es que el programa desarrollado en C++ logre capturar y ejecutar con éxito las características y los mecanismos de jugabilidad, que definen al juego en su versión de Android, priorizando siempre la calidad del diseño de software y la lógica de programación, antes que la parte visual.

El proyecto se centra en la creación de un software, en C++, del juego homónimo empleando los principios de la programación orientada a objetos (POO) y los principios SOLID. El juego cuenta con los objetos: vehículos, tablero, pasajeros, entre otros, los cuales con sus propias responsabilidades e interacciones permiten la jugabilidad de Bus Fever Party!. El software deberá implementar las reglas fundamentales del juego: gestión del tablero congestionado, movimiento restringido de vehículos, flujo de pasajeros desde una fila estática hacia la fila rotatoria, recolección de pasajeros por color y manejo de zonas de parqueo limitadas y todas las demás reglas que el grupo identifique.

A lo largo del desarrollo del proyecto, se hará uso de los cuatro pilares de la POO: abstracción, encapsulamiento, herencia y polimorfismo. Para gestionar el flujo y las reglas, se requiere el uso de una arquitectura basada en el patrón MVC (Modelo-Vista-Controlador). El proyecto también contempla la documentación técnica (diagramas de clases, documentación CRC y documentación en general: Doxygen) y la implementación de pruebas unitarias para asegurar el funcionamiento y el cumplimiento de las funcionalidades.

Para la aprobación del proyecto, el estudiante debe entregar obligatoriamente el diagrama de clases, la definición formal de las clases y sus respectivos archivos de implementación (.h y .cpp). El programa debe ser completamente funcional, permitiendo la ejecución del flujo completo del juego y superando de manera satisfactoria todos los tests de desarrollo establecidos por los miembros del equipo. Adicionalmente, se otorgará un punto de bonificación sobre la nota final a aquellos estudiantes que desarrollen, de manera autónoma, una interfaz gráfica de usuario (GUI) utilizando librerías externas para mejorar la experiencia visual del juego.

El presente documento está organizado en cinco secciones: descripción del problema, que contextualizan el reto y las mecánicas del juego; los requisitos técnicos, donde se detallan la arquitectura MVC, la persistencia de datos y los principios de programación; los entregables,

que especifican los diagramas de clases y el código a presentar; y finalmente, las secciones de bonificaciones y criterios de evaluación, que establecen las pautas para la calificación y los incentivos por funcionalidades extra como la interfaz gráfica GUIs.

2. Descripción del Problema

El juego consiste en un tablero (grid) donde diversos vehículos de colores están bloqueados por el tráfico y en un sistema de filas donde esperan los pasajeros. El objetivo es mover los vehículos estratégicamente hacia las zonas de parqueo para permitir que recojan a los pasajeros de su mismo color, quienes llegan a la posición de embarque a través de una fila estática y una fila rotatoria. Adicionalmente, el jugador debe tener en cuenta que el bus a mover no choque con otro bus.

El juego se desarrolla en un entorno de tráfico congestionado representado por un tablero de celdas (grid). En este escenario, existen vehículos de distintos tamaños, incluyendo carros (pequeños) y buses (medianos y grandes) que ocupan dos o más celdas respectivamente. Una restricción fundamental del diseño es que cada vehículo posee una orientación fija: los carros y buses pueden moverse únicamente de forma vertical u horizontal, según su disposición inicial, desplazándose hacia adelante siempre que el camino esté despejado de otros vehículos u obstáculos.

Los pasajeros avanzan por dos estructuras encadenadas antes de abordar:

- **Fila estática:** contiene la secuencia completa de pasajeros, con su color y el orden preciso en que aparecen. No rota. Solo su primer pasajero puede salir, y lo hace únicamente para ocupar un lugar libre en la fila rotatoria.
- **Fila rotatoria:** estructura circular de capacidad fija que se llena inicialmente con los primeros pasajeros de la fila estática. Gira una posición a la vez y cuenta con una posición de embarque, frente a las zonas de parqueo. Cada vez que un pasajero abandona la fila rotatoria, el primer pasajero de la fila estática (si queda alguno) ocupa el lugar libre.

El recorrido de todo pasajero es: fila estática → fila rotatoria → vehículo.

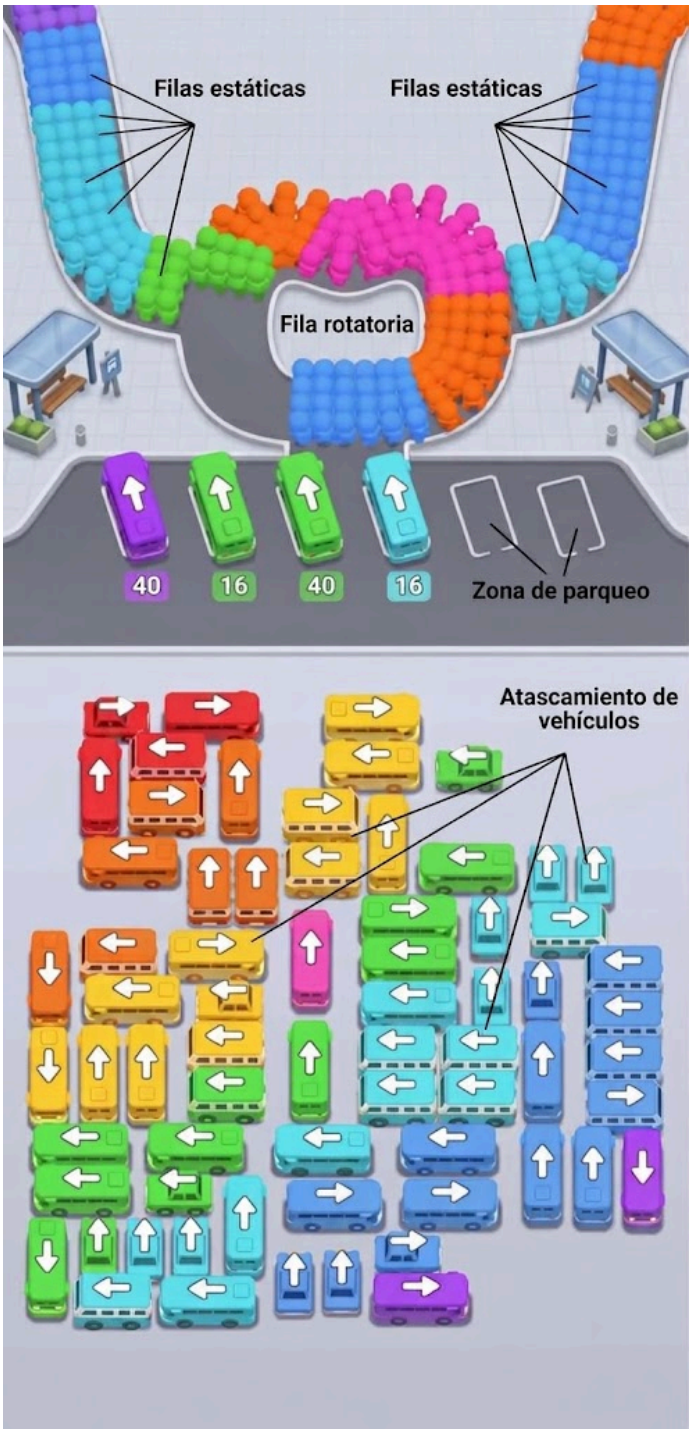


Figura 2: Descripción de los elementos principales de Bus Fever Party! Fuente: Elaboración propia a partir de una captura de pantalla del juego.

La configuración inicial del juego se carga íntegramente desde un archivo plano, el cual mantiene la persistencia del sistema. Este archivo define la estructura completa del tablero e incluye: la posición, orientación, tamaño y capacidad de cada vehículo (carros y buses), el color correspondiente a cada unidad, y la secuencia exacta de pasajeros, especificando su color y el orden preciso en que aparecen en la fila estática. Es responsabilidad del estudiante crear de forma manual el archivo, y el programa debe ser capaz de leer el archivo y transformar esta configuración externa en la estructura de objetos interna del programa.

La interacción con el programa se realizará a través de la consola. Cada vehículo en el tablero deberá estar identificado (por ejemplo, mediante números o etiquetas), permitiendo al usuario realizar la selección de carro o bus. Al presionar Enter tras seleccionar un vehículo, este se desplazará automáticamente siguiendo su eje de movimiento hasta alcanzar la zona de parqueo.

Cada vez que un vehículo llega a una zona de parqueo, el sistema ejecuta automáticamente este ciclo:

1. Se examina el pasajero ubicado en la posición de embarque de la fila rotatoria. Si hay un vehículo estacionado del mismo color con asientos libres, el pasajero aborda y el primer pasajero de la fila estática ocupa el lugar libre.
2. Si ningún vehículo puede recibirlo, la fila rotatoria gira una posición.
3. El ciclo se repite hasta que ningún pasajero de la fila rotatoria tenga un vehículo estacionado de su color con asientos libres, o hasta que no queden pasajeros.

Cuando un vehículo completa su capacidad, abandona la zona de parqueo y la deja libre.

El tablero cuenta con cuatro zonas de parqueo restringidas. Estas áreas actúan como puntos de recogida de pasajeros. El jugador debe gestionar sus movimientos con cautela, ya que, si las cuatro zonas de parqueo se llenan completamente sin que sea posible realizar recoger a los pasajeros restantes, el flujo del tráfico se bloqueará definitivamente y el jugador perderá la partida.

2.1. Requerimientos del Juego

Objetivo del Jugador

Mover estratégicamente los vehículos para permitir que recojan a los pasajeros que esperan en fila. El jugador gana cuando todos los pasajeros han abordado un vehículo. Pierde si las cuatro zonas están ocupadas y ningún pasajero de la fila rotatoria puede abordar alguno de los

vehículos estacionados, pues el tráfico queda bloqueado definitivamente.

Requisitos de los Vehículos

Existen carros (pequeños) y buses (medianos y grandes) que ocupan dos o más celdas. Tienen una orientación fija y solo pueden moverse de forma vertical u horizontal según su disposición inicial. Cada vehículo tiene un color y una capacidad máxima de pasajeros.

Requisitos del Tablero, Zonas de Parqueo y Filas de Pasajeros

El tablero (grid) debe gestionar colisiones. Cuenta con 4 zonas de parqueo; un vehículo ocupa una hasta completar su capacidad y entonces la libera. Los pasajeros vienen en una secuencia exacta de colores en la fila estática. La fila rotatoria tiene capacidad fija, gira una posición a la vez y se rellena desde la fila estática cada vez que un pasajero aborda. El sistema debe validar automáticamente la coincidencia de colores entre el pasajero en la posición de embarque y los vehículos estacionados.

Persistencia

La configuración (dimensiones del tablero; vehículos con su posición, orientación, color y capacidad; y la fila estática de pasajeros) se carga desde un archivo plano (.txt).

2.2. Reglas del Juego

- Los vehículos solo se mueven sobre su eje (horizontal/vertical) y hacia adelante, siempre que el camino esté despejado de otros vehículos u obstáculos.
- Un vehículo que sale del tablero ocupa una zona de parqueo libre. Si no hay ninguna libre, no puede salir.
- Los pasajeros avanzan siempre en el orden fila estática → fila rotatoria → vehículo.
- El pasajero en la posición de embarque aborda automáticamente si hay un vehículo estacionado de su mismo color con asientos libres.
- Si el pasajero no puede abordar, la fila rotatoria gira una posición. Un pasajero que no coincide no bloquea a los demás.
- Cada vez que un pasajero sale de la fila rotatoria, el primer pasajero de la fila estática (si queda alguno) ocupa el lugar libre.
- Un vehículo que completa su capacidad abandona la zona de parqueo y la deja libre.
- El jugador gana cuando todos los pasajeros han abordado. Pierde si las 4 zonas de parqueo están ocupadas y ningún pasajero de la fila rotatoria puede abordar ningún vehículo estacionado.

3. Requisitos Técnicos

3.1. Arquitectura (Patrón MVC)

El proyecto debe estar estrictamente dividido en tres capas:

- **Modelo (Model):** Gestión la información inicial de la partida, de los niveles y puede almacenar dónde quedó la partida.
- **Vista (View):** Clase encargada exclusivamente de la representación visual (Interfaz de consola o gráfica).
- **Controlador (Controller):** Contiene la lógica del juego, comprende las reglas del negocio, hace de intermediario que procesa las entradas del usuario y coordina los cambios en el modelo y la actualización de la vista.

3.2. Persistencia (Archivo Plano)

El estado inicial de los niveles debe cargarse desde un archivo .txt. El formato sugerido incluye:

- Dimensiones del tablero (N x M).
- Definición de vehículos (Tipo, Color, X, Y, Orientación, Largo, Capacidad).
- Fila estática de pasajeros: grupos consecutivos de un color, seguidos de un grupo de otro color, y así sucesivamente, en el orden exacto en que salen.

3.3. Conceptos de POO y SOLID

- **Abstracción y Encapsulamiento:** Uso correcto en el diseño de clases y la programación.
- **Herencia y Polimorfismo:** Jerarquía de vehículos y pasajeros.
- **SOLID:** Aplicación de Responsabilidad Única, Abierto/Cerrado, Sustitución de Liskov, Segregación de Interfaz e Inversión de Dependencias.

4. Entregables

Para la aprobación del proyecto, el estudiante debe entregar obligatoriamente el diagrama de clases, la definición CRC de las clases y sus respectivos archivos de implementación (.h y .cpp). El programa debe ser completamente funcional, permitiendo la ejecución del flujo completo del juego y superando de manera satisfactoria todos los tests de desarrollo establecidos por la cátedra. Adicionalmente, se otorgará un punto de bonificación sobre la nota final a aquellos estudiantes que desarrollen, de manera autónoma, una interfaz gráfica de usuario (GUI) utilizando librerías externas para mejorar la experiencia visual del juego.

1. **Diagrama de Clases.**

2. **Código Fuente:** Organizado en archivos .h y .cpp
3. **Archivos de Nivel:** Al menos 3 archivos planos de niveles de dificultad creciente.
4. **Documentación Técnica:** Informe sobre la aplicación de conceptos del paradigma orientado a objetos y principios SOLID.

Primera Entrega

- Requerimientos identificados: historias de usuario, criterios de aceptación y tareas de implementación.
- Diagrama de clases del proyecto.
- Archivos de Nivel: Al menos 3 archivos planos de niveles de dificultad creciente.
- Archivos de definición de las clases, implementación completa y documentada (.h) .
- Primera versión del modelo de concepto funcional del programa.

Segunda Entrega

- Verificación de la aplicación de POO (Abstracción, Encapsulamiento, Herencia, Polimorfismo) y principios SOLID en el diagrama de clases y la definición de las clases.
- Diagrama de clases versión 2.0.
- Implementación completa y documentada (.h y .cpp) versión 2.0.
- Implementación de pruebas unitarias (assert o try-test) para cada método.
- Prototipo de interfaz de usuario (modo texto o menús).
- Documentación Técnica: Informe sobre la aplicación de conceptos del paradigma orientado a objetos y principios SOLID.

Puntos extra

- **Interfaz Gráfica (GUI):** Uso de librerías externas (Qt, SFML, etc.) para una interfaz visual interactiva.
- **Multijugador:** Extensión del diseño para manejar múltiples jugadores.

5. Criterios de Evaluación

Criterio	Descripción	Peso en el curso		
		R.A.1.	**R.A.2.**	**R.A.3.**
Diseño OO y SOLID	Aplicación de abstracción, encapsulamiento, herencia, polimorfismo y SOLID.	2	2	1
Arquitectura MVC	Separación de responsabilidades.	2	2	1
Funcionalidad	Carga de archivos y lógica de juego.		4	1
Testing	Manejo de test unitarios.			3
Documentación	Diagrama de clases, CRC y comentarios Doxygen.	1	1	2
	Total	**5**	**9**	**8**